Sal: Formal Reference for the Paper Revision

Operational Semantics, RA-Linearizability, and Proof Strategy

FP Launchpad

13 October 2026

Abstract

This reference accompanies *Convergence Is Not Correctness: Verifying Mergeable Replicated Data Types*. It follows the paper's operational semantics, definition of RA-linearizability, merge proof, and sequential-specification argument, using the same notation. It states the additional hypotheses needed by the checked development: a justified state invariant, evidence from the original issuer, and visibility between updates that conflict in the sequential specification. These additions are identified where they enter the definitions. Implementation equations retain concrete state equality.

How to read this reference. Sections 3–6 have the same titles and order as the paper's formal sections. The motivation and literature discussion in Sections 1–2 of the paper are not repeated. Definition and theorem numbers within sections are specific to this reference because it makes additional premises explicit. Each main result names its manuscript counterpart; Appendix A gives a compact cross-reference, followed by the optional Lean declaration map in Appendix B. Reading the main text requires no Lean knowledge.

Status. This revision follows Sal `paper1` after its concrete-equality cleanup. The source appendix links unchanged files to proof baseline `07fe525` and revised files to `paper1`; the paper baseline is Overleaf `0450d48` (8 October 2026). The generic results are machine-checked under the stated premises, which remain obligations of each datatype. The reference distinguishes the paper's unrestricted sufficient class from its extension to certified executions. The broken-set, anchored Queue and Fugue examples explain the scope of these results. The claims concern the mathematical execution model. Definitions 4.5 and 4.8 now present operation-distinct RC-NON-COMM and equal-operation self-commutation separately. Their equivalence to the combined policy laws is checked in `paper1`: the production split interfaces preserve the existing event and state domains, five merge VCs, and independent sequential-history obligations. The other manuscript comparisons retain the stated October 8 baseline.

Contents

3 Operational Semantics **3**

4 A New Definition of RA-Linearizability **4**

5 Proof Strategy **9**

6 Correctness Against a Sequential Specification **15**

A Correspondence with the Manuscript **21**

B Mechanization Traceability **22**

3 Operational Semantics

As in the paper's Section 3, the store is a labelled transition system $\mathcal{S}_{\mathcal{D}} = (\mathcal{C}, \longrightarrow)$, parameterized by

$$\mathcal{D}_\tau = \langle \Sigma, \sigma_0, \mathsf{do}, \mathsf{merge}, \mathsf{query}, \mathsf{rc} \rangle, \qquad \tau = \langle O_\tau, Q_\tau, Val_\tau \rangle.$$

Here $\mathsf{do}(\sigma, t, r, o)$ applies an update, merge takes an ancestor state and two descendant states, and $\mathsf{query}(\sigma, q)$ returns an answer. The policy $\mathsf{rc} \subseteq O_\tau \times O_\tau$ is supplied on operations. An event is $e = (t, r, o) \in \mathcal{T} \times \mathcal{R} \times O_\tau$; write $\mathsf{do}(\sigma, e)$ for $\mathsf{do}(\sigma, t, r, o)$, and $e.t, e.r, e.o$ for its components. The policy has no role in the transition rules; Section 4 uses it to order events.

Definition 3.1 (Configuration and execution)**.** A configuration is $C = \langle S, H, P, \mathsf{vis} \rangle$, where

- S maps allocated versions to pairs (σ, E) of a concrete state and an event set. Write σ_v and $E(v)$ for these components.
- H maps active replicas to their head versions.
- $P(v)$ lists the parents of v. Versions have natural-number ranks, and each parent has smaller rank. Write $v \preceq w$ for reachability from v to w, including equality.
- vis is event visibility; write $a \xrightarrow{\mathsf{vis}} b$ when a was visible to b.

Heads are allocated. Version 0 stores $(\sigma_0, \varnothing)$. Visibility endpoints occur in the union $\mathcal{E}_C$ of head event sets, and each head contains the visible predecessors of its events. Distinct events have distinct timestamps, $a \xrightarrow{\mathsf{vis}} b$ implies $a.t < b.t$, and distinct events from the same replica are visibility-comparable. Reachable stores additionally have transitive visibility and causally closed histories at every allocated version. The common-ancestor coherence property is justified below.

All rules preserve unmentioned components and map entries. In the following rules v is fresh, and its rank is larger than the rank of each listed parent. The source and target must both satisfy the configuration conditions above.

$$\textsc{Apply}\ \frac{\begin{array}{c} e = (t, r, o) \qquad S(H(r)) = (\sigma, E) \\ t \text{ is fresh in } \mathcal{E}_C \text{ and every stored history} \qquad S' = S[v \mapsto (\mathsf{do}(\sigma, t, r, o), E \cup \{e\})] \\ H' = H[r \mapsto v] \qquad P' = P[v \mapsto [H(r)]] \qquad \mathsf{vis}' = \mathsf{vis} \cup (E \times \{e\}) \end{array}}{\langle S, H, P, \mathsf{vis} \rangle \xrightarrow{\mathsf{apply}(t,r,o)} \langle S', H', P', \mathsf{vis}' \rangle}$$

$$\textsc{Merge}\ \frac{\begin{array}{c} S(H(r_1)) = (\sigma_1, E_1) \\ S(H(r_2)) = (\sigma_2, E_2) \qquad v_\top \text{ is a greatest common ancestor of } H(r_1), H(r_2) \\ S(v_\top) = (\sigma_\top, E_\top) \qquad S' = S[v \mapsto (\mathsf{merge}(\sigma_\top, \sigma_1, \sigma_2), E_1 \cup E_2)] \\ H' = H[r_1 \mapsto v] \qquad P' = P[v \mapsto [H(r_1), H(r_2)]] \end{array}}{\langle S, H, P, \mathsf{vis} \rangle \xrightarrow{\mathsf{merge}(r_1, r_2)} \langle S', H', P', \mathsf{vis} \rangle}$$

Fork takes an active source r and fresh destination r', creates a fresh child v of $H(r)$ with $S(v) = S(H(r))$, and sets $H(r') = v$. The source head is unchanged. Query at r returns $\mathsf{query}(\sigma_{H(r)}, q)$ and leaves C unchanged.

Initially only version 0 and replica 0 are allocated, $H(0) = 0$, and visibility and parent lists are empty. An execution is a finite sequence of these transitions starting at this configuration.

Changes to the paper's rules. Fresh timestamps must increase along visibility, not merely be distinct. Fork allocates a child version holding the source snapshot. The ordinary merge uses a greatest common ancestor in the precise sense below. Certified execution adds an original-issuer premise $G(e, \sigma)$ to Apply and retains evidence about the state at issuance (Definition 4.7); the unrestricted rule itself has no client guard.

Definition 3.2 (Greatest common ancestor)**.** A greatest common ancestor g of v_1, v_2 satisfies

$$g \preceq v_1 \wedge g \preceq v_2 \wedge \forall w,\ (w \preceq v_1 \wedge w \preceq v_2) \Longrightarrow w \preceq g.$$

A maximal common ancestor need only have no strictly later common ancestor; there may be several incomparable maximal common ancestors and no greatest one. The store model carries the coherence property $E(g) = E(v_1) \cap E(v_2)$ whenever an allocated greatest common ancestor exists.

Theorem 3.3 (Common-ancestor intersection; paper Lemma 3.1)**.** *Suppose parents are allocated, histories grow along parent edges, and every event has an allocated origin version containing it which is an ancestor of every version containing that event. Then for allocated g, v_1, v_2 with g their greatest common ancestor,*

$$E(g) = E(v_1) \cap E(v_2).$$

The forward inclusion follows from history monotonicity. For a shared event, its origin is common to both heads and therefore precedes g, proving the reverse inclusion. The store-invariant proof supplies this justification for the coherence field of Definition 3.2; it is not a statement about arbitrary event labels attached to an arbitrary DAG.

3.1 Multiple Merge Bases

The ordinary rule does not cover a pair with several incomparable maximal common ancestors. The following extension supplies a base for that case.

Definition 3.4 (Recursive virtual merge base)**.** For a finite set of versions T and a version w, let $M(T, w)$ be the list, in increasing rank, of maximal elements of

$$\{x \mid x \preceq w \wedge \exists v \in T,\ x \preceq v\}.$$

Write σ_v for the stored state at v, using σ_0 as the total-function default when unallocated; correctness uses allocated supports. Define the base B and scratch fold F by

$$B(T, w) = \begin{cases} \sigma_0, & M(T, w) = [], \\ F(\{v\}, \sigma_v, \eta), & M(T, w) = v :: \eta, \end{cases}$$
$$F(T, \sigma, []) = \sigma,$$
$$F(T, \sigma, v :: \eta) = F(T \cup \{v\}, \mathsf{merge}(B(T, v), \sigma, \sigma_v), \eta).$$

The virtual base of v_1, v_2 is $B(\{v_1\}, v_2)$. The widened merge step uses this state in place of σ_g in Definition 3.1, without requiring a greatest common ancestor. Only the final merged version is allocated. The scratch folds do not create replicas, events, or intermediate stored versions.

The recursion is well-founded by the size of the joint ancestor support, lexicographically with the pending list length. The execution theorem must establish correctness of these recursive scratch states as well as ordinary stored versions; replacing g by an arbitrary maximal common ancestor is not this construction.

4 A New Definition of RA-Linearizability

RA-linearizability explains each query by a sequential execution of exactly the updates visible at the queried version. Following the paper, we first define the order that this execution must respect, relative to the event set of that version, and then give its sequential specification. The formal reference retains the paper's configuration predicate and also gives the stronger full-event criterion used by the certified results.

Notation. For an event $e = (t, r, o)$, write $\mathsf{do}(\sigma, e)$ for $\mathsf{do}(\sigma, t, r, o)$, and $\pi(\sigma)$ for replaying a list. The relation rc chooses the order of conflicting payloads; vis records visibility in configuration C.

Definition 4.1 (Events, effects, and payload policy)**.** Recall the MRDT signature $\mathcal{D}_\tau$ of the operational semantics: its concrete states are Σ, its initial state is σ_0, and its updates, queries, and answers are O_τ, Q_τ, and Val_τ. An event $e = (t, r, o)$ retains its timestamp and replica identifier. Applying it is the total effector $\mathsf{do}(\sigma, e)$; $\pi(\sigma)$ is its left-fold extension to a list. A payload policy is a relation $\mathsf{rc} \subseteq O_\tau \times O_\tau$; write $a \xrightarrow{\mathsf{rc}} b$ for $\mathsf{rc}(a.o, b.o)$. It does not inspect the events' external timestamps or replica identifiers. A configuration supplies the event set $\mathcal{E}_C$ and visibility vis (with C understood). Set

$$a \parallel b \iff \neg(a \xrightarrow{\mathsf{vis}} b) \wedge \neg(b \xrightarrow{\mathsf{vis}} a), \qquad \mathrm{distinct}(a, b) \iff a.t \neq b.t.$$

Concrete raw commutation is

$$a \leftrightarrow b \quad \iff \quad \forall s \in \Sigma, \quad \mathsf{do}(\mathsf{do}(s, a), b) = \mathsf{do}(\mathsf{do}(s, b), a).$$

These are relations on full events, even though rc acts on payloads. Throughout this reference, rc denotes only that operation relation or its payload-only event lift. The generic Lean framework also accepts an independently supplied *event replay order*, which can inspect timestamps and replica IDs. That interface is named `EventOrder`, with directed predicate `replayBefore`; it is not another definition of rc. The paper1 certificates use the operation policy explicitly stated in their hypotheses. A separate event replay order cannot be substituted silently.

4.1 RA-Linearizability for MRDTs

An absorber removes a concurrent rc edge only when it belongs to the event set being ordered. This is the paper's version-local change to the linearization relation. The invariant parameter below records a further choice in the formalization: the concrete state domain on which commutation is tested.

Definition 4.2 (Invariant-scoped concrete commutation and closure)**.** For a fixed concrete state predicate $I : \Sigma \to \mathrm{Prop}$, define

$$a \leftrightarrow_I b \quad \iff \quad \forall s \in \Sigma,\ I(s) \Longrightarrow \mathsf{do}(\mathsf{do}(s, a), b) = \mathsf{do}(\mathsf{do}(s, b), a).$$

An invariant closure certificate at C supplies

$$\begin{aligned} &I(\sigma_0), \\ &\forall s, e, \quad I(s) \wedge e \in \mathcal{E}_C \Longrightarrow I(\mathsf{do}(s, e)), \\ &\forall l, a, b, \quad I(l) \wedge I(a) \wedge I(b) \Longrightarrow I(\mathsf{merge}(l, a, b)). \end{aligned}$$

Commutation here has no simultaneous-readiness or reissuability premise. Closure includes replaying an already issued event out of causal order and repeating it. A port must independently justify the predicate and its closure; it cannot exclude a reachable state merely because that state refutes an equation.

Definition 4.3 (Linearization relation and linear extension)**.** Fix a predicate $I : \Sigma \to \mathrm{Prop}$ and write $a \leftrightarrow_I b$ for commutation on states satisfying I, as defined above. For $E \subseteq \mathcal{E}_C$, use this *same* predicate in both noncommutation tests:

$$\begin{aligned} a \,\mathrm{lo}_E^{C,I}\, b \quad \iff \quad &\big(a \xrightarrow{\mathsf{vis}} b \wedge \neg(a \leftrightarrow_I b)\big) \\ &\vee \big(a \parallel b \wedge a \xrightarrow{\mathsf{rc}} b \wedge \neg\exists c \in E.\ b \xrightarrow{\mathsf{vis}} c \wedge \neg(b \leftrightarrow_I c)\big). \end{aligned}$$

Only events in E can suppress a concurrent policy edge as absorbers. For $I(s) \equiv \text{True}$ this is the raw order lo_E^C, using $\leftrightarrow$ throughout. A list π is a *linear extension* of (E, R) when it lists each event of E exactly once, lists no other event, and places a before b whenever $a, b \in E$ are distinct and aRb. We also say that π enumerates E and respects R.

For an irreflexive relation this is the paper's linear-extension definition. The checked list-order predicate compares distinct positions; it does not reject a diagonal edge by itself. Therefore use the paper's usual definition only after excluding such edges. The guarded no-chain law excludes payload self-edges, and visibility is irreflexive. The direct lifting theorem states order irreflexivity explicitly rather than requiring replay laws. Neither order is the full visibility relation. In particular, causal implementation-commuting events may be reordered by this order. Readiness adds causal constraints to implementation replay; it does not silently redefine the public semantic order.

Definition 4.4 (Sequential specification and specification conflict)**.** For an explicitly chosen update alphabet U, let labels be $\mathrm{upd}(x)$, $x \in U$, and $(q \Rightarrow v)$. A history specification Spec is a prefix-closed set of finite label lists containing the empty list. As in the paper, a labelled transition system $\mathsf{Spec} = \langle L, \Phi, \phi_0, \longrightarrow \rangle$ gives such a language: $\rho = \ell_1 \cdots \ell_n$ is admitted when there are abstract states $\phi_1, \ldots, \phi_n$ with $\phi_{i-1} \xrightarrow{\ell_i} \phi_i$. Every finite run is admitted, so the language is prefix closed. The formal interface does not assume determinism or totality; admission need not determine a unique final abstract state. Define contextual specification commutation by

$$\begin{aligned} x \leftrightarrow_S y \quad \Longleftrightarrow \quad \forall \rho, \tau, \quad & \rho \cdot [\mathrm{upd}(x), \mathrm{upd}(y)] \cdot \tau \in \mathsf{Spec} \\ & \Longleftrightarrow \rho \cdot [\mathrm{upd}(y), \mathrm{upd}(x)] \cdot \tau \in \mathsf{Spec}. \end{aligned}$$

For a fixed label projection $f : (\mathbb{N} \times \mathbb{N} \times O_\tau) \to U$, let $\mathsf{ops}_f(\pi)$ retain the original event order while mapping each update through f. The independent specification visibility relation is

$$a \ \mathrm{sv}_C^{f,\mathsf{Spec}} \ b \quad \Longleftrightarrow \quad a \xrightarrow{\mathsf{vis}} b \wedge \neg\big(f(a) \leftrightarrow_S f(b)\big).$$

Full-input specifications use $f = \mathrm{id}$. Payload projection $f(e) = e.o$ is an explicit interface choice. It cannot be assumed adequate for specifications that distinguish original enqueue or insertion identities.

Definition 4.5 (Globally guarded replay laws)**.** The replay assumptions are RC-NON-COMM, self-commutation, NO-RC-CHAIN, and COND-COMM. Write $G_{\mathrm{raw}}(a, b) \equiv a.t \neq b.t \wedge a.r \neq b.r$ for the raw event guard. The first two clauses split on equality of operation payloads, including all operation arguments:

$$\begin{aligned} & G_{\mathrm{raw}}(a, b) \wedge a.o \neq b.o \quad \Longrightarrow \\ & \qquad \big(\neg(a \leftrightarrow b) \iff a \xrightarrow{\mathsf{rc}} b \vee b \xrightarrow{\mathsf{rc}} a\big) && \text{(RC-NON-COMM)}, \\ & G_{\mathrm{raw}}(a, b) \wedge a.o = b.o \quad \Longrightarrow \quad a \leftrightarrow b && \text{(self-commutation)}. \end{aligned}$$

No-chain retains its timestamp guards:

$$a.t \neq b.t \wedge b.t \neq c.t \quad \Longrightarrow \quad \neg(a \xrightarrow{\mathsf{rc}} b \wedge b \xrightarrow{\mathsf{rc}} c).$$

For every $s \in \Sigma$, intervening event list β, and a, b, c with pairwise distinct timestamps, it also requires

$$a \xrightarrow{\mathsf{rc}} b \wedge \neg(b \leftrightarrow c) \quad \Longrightarrow \quad \mathsf{do}\big(\beta(\mathsf{do}(\mathsf{do}(s, b), a)), c\big) = \mathsf{do}\big(\beta(\mathsf{do}(\mathsf{do}(s, a), b)), c\big).$$

There is no different-replica guard on the last equation. Its absorber premise is actual noncommutation, rather than the policy edge $b \xrightarrow{\mathsf{rc}} c$.

The no-chain clause allows $a = c$ when the two adjacent timestamp guards hold; therefore a payload self-edge is not rescued by external event freshness. These global equations quantify over arbitrary concrete scratch states and intervening lists. They are not the scoped laws below. The split is equivalent to the former combined exactness law on the same guarded domain: no-chain forbids a policy self-edge, so combined exactness implies equal-payload commutation; conversely, the two payload cases recover combined exactness. Self-commutation does not assert idempotence, nor does it compare arbitrary same-replica instances. For certified datatypes use the scoped formulation of Definition 4.8.

The paper's criterion. Let an active replica r have head record (σ, E) and let $b = \mathsf{query}(\sigma, q)$. The paper's Definition "RA-linearizability" requires, for every such r and q, a list π with

$$\pi \text{ a linear extension of } (E, \mathsf{lo}_E^C), \qquad \mathsf{ops}(\pi) \cdot (q \Rightarrow b) \in \mathsf{Spec},$$

where $\mathsf{ops}(\pi) = e_1.o \cdots e_n.o$. Executions satisfy this at every configuration, and implementations at every execution. The global replay laws are assumptions of the paper's correctness argument, rather than fields of the configuration predicate.

The certified full-event interface strengthens this witness as follows.

Definition 4.6 (The full-event invariant criterion)**.** For every allocated version v with stored record (s, E) and every query q, there must exist *one* list π such that
1. π enumerates exactly E;
2. π respects $\mathrm{lo}_E^{C,I}$;
3. π respects $\mathrm{sv}_C^{\mathrm{id},\mathsf{Spec}}$;
4. $\mathsf{ops}_{\mathrm{id}}(\pi) \cdot [q \Rightarrow \mathsf{query}(s, q)] \in \mathsf{Spec}$.

The universal invariant $I(s) \equiv \mathrm{True}$ recovers the raw full-event criterion, including specification visibility. Checking every allocated version implies the corresponding guarantee for current replica heads. A projected criterion replaces the specification alphabet through f; it still preserves the specification conflicts of those projected labels. The paper's Definition "RA-linearizability" instead checks active heads, uses $\mathsf{ops}(\pi) = e_1.o \cdots e_n.o$, and omits item 3. The full-event invariant criterion retains original update identities. For a fixed label projection, requiring the same witness to preserve specification conflicts strengthens its order obligation. These are different criteria, and changing the label alphabet is an explicit interface choice.

An independent sequential simulation may use a relation, or an abstraction function $\alpha : \Sigma \to \Phi$, to relate implementation states to the *specification's* states. This does not replace concrete equality in the merge VCs, diamonds, or Definition 4.2. Universal fold soundness is one sufficient bridge. Certified Queue and RGA results instead establish appropriate original-event witnesses for represented histories. Neither implementation replay equality nor existence of an implementation fold alone establishes item 4, and a specification-legal witness must also satisfy item 3.

4.2 Canonical States and Certified Replay

The paper's "Uniqueness of the replayed state" asserts both existence of a linear extension and equality of the folds of *all* linear extensions for a supported, causally closed event set E of its order under the three global replay assumptions. Its "Canonical state" uses that assertion to define $\mathsf{can}_C(E)$, and "Canonical correctness" requires every allocated version to store this state. The checked scoped result below has a different domain: it compares legal, visibility-respecting replays at represented prefixes. It supplies uniqueness on that domain, rather than existence or uniqueness over all public-order linear extensions.

Definition 4.7 (Original issuance, certified executions, and readiness)**.** An issuance discipline is a predicate $G(e, s)$ checked at the materialized replica head from which e was minted. Mint honesty at C requires, for every $e \in \mathcal{E}_C$, a list π_e satisfying

$$\pi_e \text{ enumerates } \{p \in \mathcal{E}_C \mid p \xrightarrow{\mathsf{vis}} e\}, \qquad \pi_e \text{ respects } \mathsf{vis}, \qquad G(e, \pi_e(\sigma_0)).$$

An issued application is an actual raw store application with its original head/version evidence and $G(e, s)$; non-application steps preserve the raw fork/merge rule. A certified execution starts at the initial store and, at every transition, retains mint honesty before and after the issued step. The widened execution permits recursively synthesized virtual merge bases. It has its own certified reachability relation, with the ordinary steps as its base fragment.

A replay scope is (C, F, R), where F is its fixed eligible event set and $R(H, s)$ records representation of the already replayed prefix H. Define

$$\mathrm{Ready}(H, e) \quad \Longleftrightarrow \quad e \in F \wedge e \notin H \wedge \forall p \in F,\ p \xrightarrow{\mathsf{vis}} e \Longrightarrow p \in H.$$

A replay is legal from H by the inductive rules

$$\mathrm{Legal}(H, []), \qquad \frac{\mathrm{Ready}(H, e) \quad \mathrm{Legal}(H \cup \{e\}, \pi)}{\mathrm{Legal}(H, e :: \pi)}.$$

The issuance guard is retained as evidence about the original mint. It is not checked again when the effector is replayed at a different scratch state.

Definition 4.8 (Scoped replay and policy obligations)**.** The invariant replay laws require

$$R(H, s) \Longrightarrow I(s), \qquad R(H, s) \wedge \mathrm{Ready}(H, e) \Longrightarrow R(H \cup \{e\}, \mathsf{do}(s, e)).$$

They also require the concrete diamond $\mathsf{do}(\mathsf{do}(s, a), b) = \mathsf{do}(\mathsf{do}(s, b), a)$ whenever $R(H, s)$, both events are ready, $a \parallel b$, and neither direction of $\mathrm{lo}_F^{C,I}$ orders them. The accompanying policy laws split into

$$\begin{aligned} a.o \neq b.o \quad &\Longrightarrow \quad \big(\neg(a \leftrightarrow_I b) \Longleftrightarrow a \xrightarrow{\mathsf{rc}} b \vee b \xrightarrow{\mathsf{rc}} a\big), \\ a.o = b.o \quad &\Longrightarrow \quad a \leftrightarrow_I b. \end{aligned}$$

Both clauses quantify only over $a, b \in F$ that are concurrent, have distinct timestamps, and come from different replicas. Commutation here quantifies over *every state satisfying* I, exactly as in Definition 4.2. No-chain is required for $a, b, c \in F$ with $a.t \neq b.t$ and $b.t \neq c.t$, as in Definition 4.5. The package consists of both replay laws and policy laws.

An alternative represented-prefix commutation test instead quantifies over represented prefixes where both events are ready. That definition is distinct from $\leftrightarrow_I$; replacing the latter by readiness would make causal-pair tests potentially vacuous. The same two-clause split applies to that alternative using its represented-prefix commutation relation throughout. In either formulation, no-chain makes the split equivalent to combined policy exactness. If $F \subseteq \mathcal{E}_C$, the supported configuration's freshness and per-replica visibility laws imply the metadata guards for distinct concurrent events. They can then be left implicit, but membership and concurrency cannot. Original issuance evidence restricts which events are eligible; state reachability alone does not supply that restriction. The concrete replay diamond above remains a separate obligation, as does the sequential-history bridge in Section 6.

Theorem 4.9 (Scoped implementation replay equality)**.** *Suppose the replay laws of Definition 4.8 hold, $R(H, s)$, and π_1, π_2 enumerate the same event set A. If both lists are legal from H and each respects both* vis *and* $\mathrm{lo}_F^{C,I}$*, then*

$$\pi_1(s) = \pi_2(s).$$

Its hypotheses are scoped replay laws, not the global class of Definition 4.5. This theorem proves concrete implementation agreement. Independent sequential legality is a further obligation.

Scoped canonical state. For a scope (C, F, R), say that σ is canonical from σ_0 when there is a list π enumerating F such that

$$\mathrm{Legal}(\emptyset, \pi), \qquad \pi \text{ respects } \mathsf{vis} \text{ and } \mathrm{lo}_F^{C,I}, \qquad \pi(\sigma_0) = \sigma.$$

Under the replay laws and $R(\emptyset, \sigma_0)$, any two such states are equal by Theorem 4.9. Existence remains a witness obligation. This is the certified counterpart of the paper's canonical state; canonical correctness still means concrete equality with a replay of the version's own events. A separate bridge must show that the witness is admitted by Spec and preserves its conflicts.

5 Proof Strategy

The proof follows the manuscript's three steps: the *Eight Conditions*, the *Join Lemma*, and *Canonical Correctness of Every Reachable Configuration*. The checked argument retains these steps but makes the representation of event metadata an explicit obligation. It first proves that a merge represents the union of its inputs, then transfers that result to canonical replay, and finally lifts it through executions. The independent sequential specification is addressed in Section 6.

5.1 The Eight Conditions

The manuscript counts three replay laws and five merge equations. The three replay laws (noncommutation agreement with rc, absence of an rc chain, and conditional commutation in the presence of an absorber) are stated in Definition 4.5, with noncommutation agreement split into operation-distinct exactness and equal-operation self-commutation; the invariant-scoped route has its own replay and policy obligations. The join induction below does not silently infer metadata reconstruction from these three laws. It consumes the five equations together with the explicit representation and replay supply stated below. In particular, an acyclic semantic order alone is insufficient for its choice of the last event.

Abbreviations. Write $\mathsf{do}(s, e)$ for the effect of the original event e and $\mathsf{merge}(l, a, b)$ for three-way merge. The context C supplies the events and visibility of the configuration; it is held fixed during the join proof. Use the raw set-relative order lo_E^C of Definition 4.3, with all-state concrete commutation $\leftrightarrow$. Its absorber is searched for in E, so the order is recomputed after an event is removed. The invariant-scoped history order has its separately stated commutation test; it is not substituted into the raw join induction.

Definition 5.1 (Indexed concrete representation)**.** For each event context C, a representation is a predicate $R_C(E, s)$ on event sets E and concrete states $s \in \Sigma$. An event context records $\mathcal{E}_C$ and its visibility relation, as supplied by a configuration. We write $R(C, E, s)$ interchangeably with $R_C(E, s)$. The assertion $R(C, E, s)$ relates the *entire concrete state* s to a specific event set E in

context C. It need not be defined as reachability, and it is not inferred from the desired query answer. Define

$$\begin{aligned}\mathsf{Supp}_C(E) &\iff E \subseteq \mathcal{E}_C,\\ \mathsf{Unique}(R) &\iff \forall C, E, s, t.\ R(C,E,s) \land R(C,E,t) \Longrightarrow s = t.\end{aligned}$$

A history is causally closed when

$$\mathsf{CausalClosed}_C(E) \iff \forall a, b.\ a \xrightarrow{\mathsf{vis}} b \land b \in E \Longrightarrow a \in E.$$

An enumeration $\pi \equiv E$ is a finite list without repetitions whose members are exactly E. Thus every finite-set assumption below is an explicit enumeration assumption.

Why the reconstruction past changes. The manuscript uses the principal downset of non-commuting visibility. A concrete representation may need a commuting predecessor's identifier or anchor to reconstruct an event. Accordingly the checked contract chooses an explicit dependency relation M_C, and writes $\downarrow_M e$ for its principal downset. Commutation is the concrete relation $\leftrightarrow$. All representation equations below use concrete state equality.

Definition 5.2 (Metadata dependencies)**.** For every context C, choose a relation M_C on events satisfying

$$\begin{aligned}M_C(a,b) &\Longrightarrow a \xrightarrow{\mathsf{vis}} b,\\ a \xrightarrow{\mathsf{vis}} b \land \neg(a \leftrightarrow b) &\Longrightarrow M_C(a,b).\end{aligned}$$

It may retain additional visible predecessors which commute. Define

$$\begin{aligned}\mathsf{Closed}_{M_C}(E) &\iff \forall a, b.\ M_C(a,b) \land b \in E \Longrightarrow a \in E,\\ \downarrow_M e &= \{x \mid x = e \ \lor\ M_C^+(x,e)\},\\ \mathsf{Max}^{\mathsf{lo}}_{C,E}(e) &\iff \forall x \in E \setminus \{e\}.\ \neg \mathsf{lo}^C_E(e,x),\\ \mathsf{Max}^{M}_{C,E}(e) &\iff \forall x \in E \setminus \{e\}.\ \neg M_C(e,x).\end{aligned}$$

The two maximality conditions are separate. If $E \subseteq U$ is M_C-closed and e is M_C-maximal in U, then $E \setminus \{e\}$ remains M_C-closed. Semantic maximality alone does not justify this step. For the embedded sequence and anchored queue ports, $M_C = \mathsf{vis}$; their reconstruction past retains all causal metadata.

Definition 5.3 (Five concrete merge VCs)**.** The following conditions are universally quantified over the displayed contexts, histories, events and states. Define the common hypotheses

$$\begin{aligned}A_C(E) &:= \mathsf{Supp}_C(E) \land \mathsf{Closed}_{M_C}(E),\\ \mathsf{Admissible}_C(E_1,E_2,e) &:= \mathsf{Trans}(\mathsf{vis}) \land \mathsf{Irrefl}(\mathsf{vis}) \land A_C(E_1) \land A_C(E_2)\\ &\quad \land \mathsf{Max}^{\mathsf{lo}}_{C,E_1\cup E_2}(e) \land \mathsf{Max}^{M}_{C,E_1\cup E_2}(e).\end{aligned}$$

Within each condition, abbreviate $R_C(E,s) := R(C,E,s)$, $D :=\downarrow_M e$ and $E^- := E \setminus \{e\}$. These abbreviations compress only repeated hypotheses.

VC1. *Merge commutativity.* From

$$A_C(E_1),\ A_C(E_2),\ R_C(E_1 \cap E_2, l),\ R_C(E_1, a),\ R_C(E_2, b)$$

conclude $\mathsf{merge}(l,a,b) = \mathsf{merge}(l,b,a)$. This equation is required on represented triples. The manuscript asks for the stronger equation on arbitrary states; that stronger equation discharges this restricted obligation.

VC2. *Initial branch.* From $A_C(E)$ and $R_C(E, s)$ conclude

$$\mathsf{merge}(\sigma_0, \sigma_0, s) = s.$$

This closes the induction when one branch and its intersection are empty.

VC3. *Local redistribution.* Assume $\mathsf{Admissible}_C(E_1, E_2, e)$, $e \in E_1 \setminus E_2$, and *all* the representation premises

$$R_C(E_1 \cap E_2, l), \quad R_C(D^-, B), \quad R_C(E_1^-, t), \quad R_C(E_2, b),$$
$$R_C(D, \mathsf{do}(B, e)), \quad R_C(E_1, \mathsf{merge}(B, t, \mathsf{do}(B, e))),$$
$$R_C((E_1 \cup E_2)^-, \mathsf{merge}(l, t, b)).$$

Then

$$\mathsf{merge}(l, \mathsf{merge}(B, t, \mathsf{do}(B, e)), b) = \mathsf{merge}(B, \mathsf{merge}(l, t, b), \mathsf{do}(B, e)).$$

The event occurs in one branch only. The equation moves its effect from inside the reconstructed branch to outside the merge of the remaining events.

VC4. *Shared redistribution.* Assume $\mathsf{Admissible}_C(E_1, E_2, e)$, $e \in E_1 \cap E_2$, and

$$R_C((E_1 \cap E_2)^-, t_0), \quad R_C(D^-, B),$$
$$R_C(E_1^-, t_1), \quad R_C(E_2^-, t_2), \quad R_C(D, \mathsf{do}(B, e)),$$
$$R_C(E_1 \cap E_2, \mathsf{merge}(B, t_0, \mathsf{do}(B, e))),$$
$$R_C(E_1, \mathsf{merge}(B, t_1, \mathsf{do}(B, e))), \quad R_C(E_2, \mathsf{merge}(B, t_2, \mathsf{do}(B, e))),$$
$$R_C((E_1 \cup E_2)^-, \mathsf{merge}(t_0, t_1, t_2)).$$

Then

$$\mathsf{merge}\big(\mathsf{merge}(B, t_0, \mathsf{do}(B, e)),$$
$$\mathsf{merge}(B, t_1, \mathsf{do}(B, e)), \mathsf{merge}(B, t_2, \mathsf{do}(B, e))\big)$$
$$= \mathsf{merge}\big(B, \mathsf{merge}(t_0, t_1, t_2), \mathsf{do}(B, e)\big).$$

The event occurs in both branches and their intersection. The equation moves the three reconstructed occurrences outside the merge, leaving one.

VC5. *Causal delta.* Assume transitive, irreflexive visibility, $A_C(U)$, $e \in U$, both $\mathsf{Max}^{\mathsf{lo}}_{C,U}(e)$ and $\mathsf{Max}^{M}_{C,U}(e)$, and

$$R_C(U^-, s), \quad R_C(D^-, B), \quad R_C(D, \mathsf{do}(B, e)), \quad R_C(U, \mathsf{do}(s, e)).$$

Then $\mathsf{merge}(B, s, \mathsf{do}(B, e)) = \mathsf{do}(s, e)$. After redistribution this equation removes the last merge: the event is applied once to the represented remainder.

The represented smaller-union merge in VC3 and VC4 is obtained by a *strictly smaller* induction call. Its presence is not a premise asserting the desired join theorem for the original union. The represented side reconstructions are also derived on proper subsets, or by VC5 when a side equals the whole union.

Reusable verification templates

The five equations are a reusable interface, rather than five independent proof scripts for each datatype. A template fixes a family of implementation algebras and proves that a declarative instance of that family supplies Definition 5.3. In the checked interface, an instance retains the

datatype D, public operation policy P, concrete representation R and metadata dependencies M. Its soundness statement has the shape

$$\mathsf{Input}(D, P, R, M) \Longrightarrow \mathsf{MergeVCs}(P, R, M).$$

The conclusion contains exactly the five equations and their represented-state premises. The instance cannot replace R by a weaker query relation or replace P or M by a convenient alternative.

The reusable families include commuting updates with empty reconstruction past, policy-controlled replay, immutable certified records, ordered records, products with monotone components, and ordered records with an archive. A query extension preserves the update and merge algebra; its transport rule therefore preserves the five VCs. This transport alone proves no assertion about the new query's independent sequential specification.

The datatype author chooses the family and identifies its implementation maps. For the ordinary and efficient OR-sets, proof-free annotations record the event comparison and, optionally, the record type and carrier/update/birth/kill maps. The frontend selects witness or mask replay, constructs the templates, and proves their finite laws from registered definitions. Embedded and Sided RGA identify eight ordered-record maps, raw-equation annotations and chain/code mappings; generic list, comparator and prefix-code laws construct their finite kits from checked raw recursive equations, and the frontend projects their existing issuance and replay contracts. Anchored Queue and Peritext reuse this route, including the Sided Core and RichCore transports. For Fugue, an archive template derives the finite kit from live-list and archive projection equations. Compositional prefix-code laws derive coordinate injection, while generic issuance lemmas propagate record validity and creator evidence. The datatype still supplies its code specialization, issuer lookup facts and the connection to its unchanged issuance check. Other families still supply their finite collection of algebraic laws and evidence projections explicitly. Here *finite* describes the number of obligations, not a finite test domain: each law is universally quantified over its typed inputs. Evidence projections may concern every represented history. The annotations do not define the independent sequential specification or prove its correctness bridge.

The verification command applies the generic soundness theorem and searches registered declarative instances. Its registration interface rejects completed VC and join contracts as inputs. Missing family laws remain typed proof goals; there is no fallback to an existing datatype VC theorem. The resulting proof term is checked by Lean. Automation thus packages a proved algebraic argument; it does not infer correctness from executions sampled by a test harness.

To obtain the Join Lemma, the author still supplies uniqueness, initial representation, finite enumeration, and replay supply as stated below. To obtain RA-linearizability, the independent sequential bridge in Section 6 remains necessary. In particular, applying an ordered-record template to Fugue's implementation leaves the sequential-specification obstruction of Example 6.6 unresolved.

Definition 5.4 (Replay supply and a peel witness)**.** At a fixed context C, replay supply consists of both clauses:

(i) For every E and π, if $\pi \equiv E$ and $\mathsf{Supp}_C(E)$, some s satisfies $R_C(E, s)$.

(ii) For every represented, supported, nonempty, M_C-closed U, there exist $e \in U$ and states s, B satisfying

$$\mathsf{Max}^{\mathsf{lo}}_{C,U}(e), \qquad \mathsf{Max}^{M}_{C,U}(e),$$
$$R_C(U^{-}, s), \quad R_C((\downarrow_M e) \setminus \{e\}, B),$$
$$R_C(\downarrow_M e, \mathsf{do}(B, e)), \quad R_C(U, \mathsf{do}(s, e)).$$

The second clause is the *peel witness*. It supplies the represented predecessor state and the represented effect of appending the original event both to its metadata past and to the remainder. It does not assume that merging preserves representation. Its existence is a separate datatype obligation; acyclicity of lo_U^C alone supplies neither metadata maximality nor the four representation assertions.

5.2 The Join Lemma

The manuscript's canonical-state equation becomes a representation statement until the correspondence with canonical replay has been proved.

Theorem 5.5 (Join: finite concrete representation induction)**.** *Assume the five VCs,* $\mathsf{Unique}(R)$*, and*

$$\forall C, E, s.\ R(C, E, s) \Longrightarrow R(C, \varnothing, \sigma_0),$$
$$\forall C, E, s.\ R(C, E, s) \Longrightarrow \exists \pi.\ \pi \equiv E.$$

Fix a context C with transitive, irreflexive visibility and replay supply. For all E_1, E_2, l, a, b, if $A_C(E_1)$, $A_C(E_2)$ and

$$R_C(E_1 \cap E_2, l), \qquad R_C(E_1, a), \qquad R_C(E_2, b),$$

then

$$R_C(E_1 \cup E_2, \mathsf{merge}(l, a, b)).$$

Proof structure. Induct strongly on the length of a duplicate-free enumeration of $E_1 \cup E_2$. An empty side uses uniqueness, the initial-representation hypothesis and VC2, with VC1 if the right side is empty. Otherwise replay supply chooses a peel witness. Removing its event preserves metadata closure by M_C-maximality. Supply gives represented predecessor states for the branches and their intersection. The smaller induction calls establish side reconstruction and the smaller-union merge representation. Uniqueness identifies the reconstructed sides with the original sides; VC3 or VC4 then brings the event outside. VC5 and the peel witness's represented update reconstruct the union. Every recursive call has a strictly smaller event set; none assumes representation preservation of an unresolved merge. □

Remark 5.6 (The full-causal-closure join contract)*.* The checked derived contract quantifies over fully causally closed E_1, E_2, not over all merely conflict-closed histories. To obtain it from Theorem 5.5, require replay supply for every C, E_1, E_2, a, b with transitive, irreflexive visibility, supported sides and $R_C(E_1, a), R_C(E_2, b)$. This is the quantifier order of the supply hypothesis in the checked derived theorem. Causal closure implies M_C-closure because $M_C \subseteq \mathsf{vis}$. Hence the theorem gives the concrete *representation join* contract

$$\begin{aligned} &\mathsf{Supp}_C(E_1),\ \mathsf{Supp}_C(E_2),\quad \mathsf{CausalClosed}_C(E_1),\ \mathsf{CausalClosed}_C(E_2),\\ &R_C(E_1 \cap E_2, l),\ R_C(E_1, a),\ R_C(E_2, b) \quad\Longrightarrow\quad R_C(E_1 \cup E_2, \mathsf{merge}(l, a, b)), \end{aligned}$$

under transitive, irreflexive visibility. This contract includes intermediate representation evidence which canonical query agreement by itself does not provide.

5.3 Canonical Correctness of Every Reachable Configuration

Theorem 5.7 (Canonical correctness of certified executions)**.** *Fix an internal replay policy whose directed event relation is ρ. This auxiliary policy supplies concrete replay witnesses; it is separate*

from the public payload policy and from the revised invariant-scoped history order. Let $K_\rho(a,b) := \rho(a,b) \vee \rho(b,a)$*. Its set-relative order is*

$$\begin{aligned}\mathsf{lo}_E^{\rho,C}(a,b) \iff\ & (a \xrightarrow{\mathsf{vis}} b \wedge K_\rho(a,b)) \\ & \vee \big(\neg(a \xrightarrow{\mathsf{vis}} b) \wedge \neg(b \xrightarrow{\mathsf{vis}} a) \wedge \rho(a,b) \\ & \qquad \wedge \neg\exists c \in E.\ b \xrightarrow{\mathsf{vis}} c \wedge K_\rho(b,c)\big).\end{aligned}$$

Define

$$\mathsf{IsCan}_C^\rho(E,s) \iff \exists \pi.\ \pi \equiv E \wedge \mathsf{Respects}(\pi, \mathsf{lo}_E^{\rho,C}) \wedge \pi(\sigma_0) = s.$$

Suppose, for every configuration C *satisfying original mint honesty, the following full-causal-closure canonical join contract holds:*

$$\begin{gathered}\mathsf{Trans}(\mathsf{vis}),\ \mathsf{Irrefl}(\mathsf{vis}), \quad \mathsf{Supp}_C(E_1),\ \mathsf{Supp}_C(E_2), \\ \mathsf{CausalClosed}_C(E_1),\ \mathsf{CausalClosed}_C(E_2), \\ \mathsf{IsCan}_C^\rho(E_1 \cap E_2, l), \quad \mathsf{IsCan}_C^\rho(E_1, a), \quad \mathsf{IsCan}_C^\rho(E_2, b) \\ \implies \quad \mathsf{IsCan}_C^\rho(E_1 \cup E_2, \mathsf{merge}(l,a,b)).\end{gathered}$$

Every ordinary or recursive-virtual issuance-certified execution from the initial configuration then has $\mathsf{IsCan}_C^\rho(E,s)$ *at every allocated version* $C.S(v) = (s,E)$*. This theorem concludes existence of a respecting replay. To recover the manuscript's canonical-state equality, additionally establish uniqueness of such folds under the internal policy; write their common state as* $\mathsf{can}_C^\rho(E)$*; then the join contract reads* $\mathsf{can}_C^\rho(E_1 \cup E_2) = \mathsf{merge}(\mathsf{can}_C^\rho(E_1 \cap E_2), \mathsf{can}_C^\rho(E_1), \mathsf{can}_C^\rho(E_2))$*. The execution theorem itself needs only the displayed witness contract. Its event sets are supported and causally closed; visibility is transitive and irreflexive. Moreover, if the store invariant and this canonical configuration invariant hold, its recursive virtual base for versions* (s_1,E_1) *and* (s_2,E_2) *is canonical for* $E_1 \cap E_2$*.*

To derive the displayed join hypothesis from the datatype equations, the five VCs, uniqueness, initial representation, finite enumeration and replay supply yield representation join. A port additionally proves

$$R_C(E,s) \quad \iff \quad \mathsf{IsCan}_C^\rho(E,s)$$

for supported, causally closed histories in each mint-honest context with transitive, irreflexive visibility. The backward implication makes the three canonical inputs represented; representation join constructs the union; the forward implication makes its result canonical. The embedded ports have exactly this correspondence. Other ports can use stronger represented canonical-state certificates rather than asserting this equivalence generally.

The execution induction carries the store invariant together with canonicality. The initial version replays the empty list to σ_0. A query preserves that invariant and a fork copies it. An application appends its freshly issued event to a replay of the previous head; the new event has no successor in the resulting event set, so this list respects its set-relative order. The merge cases use the canonical join contract. Ordinary merges use the actual greatest common ancestor's intersection history. This event-intersection coherence is an admissibility property of the configuration in Definition 3.1; for operational stores it is proved from the store invariant, rather than from graph ancestry alone. Recursive virtual merges use a nested induction on finite ancestry support: the inner fold represents the union of its inputs, and each recursively computed base represents their intersection. Original issuance guards remain at original mint states. Neither intermediate peeling nor scratch computation requires reissuing an event. This theorem is a merge-and-replay guarantee;

legality and specification visibility of one independent sequential word are additional obligations of the final correctness bridge.

6 Correctness Against a Sequential Specification

Canonical correctness explains each stored state by a fold through the implementation's own do. Together with replay determinacy it gives convergence. To check the answers against a sequential specification, that same sequence must also be admitted by the independent language Spec of Definition 4.4. This section states the paper's sequential lifting theorem, distinguishes its literal RA criterion from the criterion retaining specification conflicts, and gives an additional route for guarded operations. Write $\pi(\sigma_0)$ for the merge-free fold of do along π, $\mathsf{ops}_f(\pi)$ for its projected update labels, and lo_E^C for the raw implementation order of Definition 4.3. The scoped bridge below also uses this raw order. Ports using the invariant order $\mathsf{lo}_E^{C,I}$ instead supply their own exact history witnesses.

Example 6.1 (A convergent implementation which is not a set)**.** Take a one-element observed-remove set, represented by addition timestamps, and change removal to the identity. Addition inserts its timestamp, and the query reports whether the timestamp set is nonempty. Keep the merge

$$\mathsf{merge}(l, a, b) = (l \cap a \cap b) \cup (a \setminus l) \cup (b \setminus l).$$

Every pair of implementation updates commutes. With empty rc, lo_E^C is empty; replicas with the same events converge. At one replica, perform

$$\mathsf{add}\ ;\ \mathsf{rem}\ ;\ (\mathsf{rd} \Rightarrow \mathsf{true}).$$

The implementation retains the addition timestamp, while the ordinary set specification admits only $\mathsf{rd} \Rightarrow \mathsf{false}$ after this sequence. It therefore fails the unrestricted sequential premise below.

There is a further distinction concerning the paper's literal RA definition. The word $[\mathsf{rem}, \mathsf{add}]$ enumerates the same events, respects the empty implementation order, and explains the returned answer in the set language. In fact this implementation satisfies the literal criterion on every raw ordinary execution. In the set specification, addition and removal do not contextually commute. Specification visibility requires the causal addition-before-removal order, so the strengthened criterion rejects this execution. The example refutes an identification of the two criteria; it does not refute a lifting theorem with an unrestricted sequential premise.

6.1 The paper's sequential lifting theorem

Theorem 6.2 (Lifting canonical correctness to literal RA)**.** *Assume lo_E^C has no diagonal edge on each stored event set E. Let every stored version (s, E) of C have an exact canonical witness: a list π enumerating E, respecting lo_E^C, with $\pi(\sigma_0) = s$. Suppose also that, for every event list π and every query q,*

$$\mathsf{ops}_f(\pi) \cdot [q \Rightarrow \mathsf{query}(\pi(\sigma_0), q)] \in \mathsf{Spec}, \qquad f(e) = e.o. \tag{1}$$

Then every stored version, hence every replica head, satisfies the paper's literal RA criterion with respect to Spec*.*

This corresponds to Theorem 6.1 of the manuscript. Exact canonical witnesses already respect the displayed raw order, so the lifting step needs no additional replay laws. The conclusion here omits the extra specification visibility clause in Definition 4.6.

Proof. Choose the exact canonical witness of the stored version. Substitute $\pi(\sigma_0) = s$ into Equation (1). The same word provides the enumeration, implementation order, and admitted query answer. The chosen word works for every query. □

The premise ranges over arbitrary event lists, independently of any execution or version. It compares two sequential machines and does not mention merge. It does not claim that every permutation respects lo_E^C.

Discharging the premise. For a deterministic sequential specification with total updates and total queries that leave the abstract state unchanged, $\mathsf{Spec} = \langle L, \Phi, \phi_0, \to \rangle$, an abstraction function $\alpha : \Sigma \to \Phi$ suffices if it satisfies

$$\begin{aligned} &\alpha(\sigma_0) = \phi_0, \\ &\alpha(\sigma) \xrightarrow{e.o} \phi' \quad \Longrightarrow \quad \alpha(\mathsf{do}(\sigma, e.t, e.r, e.o)) = \phi', \\ &\alpha(\sigma) \xrightarrow{q \Rightarrow b} \alpha(\sigma) \quad \Longrightarrow \quad \mathsf{query}(\sigma, q) = b. \end{aligned}$$

The last two clauses quantify over every concrete state, event, and query. Totality supplies the displayed transitions; determinism fixes their targets and answers. There is no issuance, freshness, or invariant premise. Induction on π relates its implementation fold to the abstract state reached by $\mathsf{ops}_f(\pi)$, and the query clause proves Equation (1). The checked simulation permits a relation $R \subseteq \Sigma \times \Phi$ in place of a function: initially $R(\sigma_0, \phi_0)$, each event preserves R, and related states agree on all queries. The functional case takes $R(\sigma, \phi)$ to mean $\alpha(\sigma) = \phi$.

Natural abstract states. The grow-only-set specifications use $S \subseteq A$, initially $\varnothing$, with $S \xrightarrow{\mathsf{add}(a)} S \cup \{a\}$ and observation S (or its characteristic function). Counter specifications use $z \in \mathbb{Z}$, initially 0, with $z \xrightarrow{u} z + \delta(u)$ and observation z. These replace earlier operation-log presentations without changing their history languages, including queries interspersed with updates. More generally, two total deterministic sequential machines have equal admitted languages when a map between their states preserves initialization, every update, and every query. Forward simulation maps a run; the reverse argument follows the same updates from the source initial state. Surjectivity of the map is unnecessary. This is a sequential-state simplification, not a quotient of implementation states or a change to the five VCs.

For an observed-remove set take $\alpha(\sigma) = \{a \mid \exists t.\ (a, t) \in \sigma\}$. Addition gains a, removal loses a, and membership queries agree. For the implementation of Example 6.1, removal leaves a nonempty state unchanged while the abstract transition removes its element. No abstraction function can satisfy these clauses: initialization and addition force the resulting state to abstract to presence, while removal of that same unchanged state forces it to abstract to absence. This contradiction explains the failure of the sequential premise.

For both set implementations, a shared template now derives these projection laws from record-level actions and their raw update equations. An addition may replace tags for its own element, but must retain records for other elements; removal filters all records for the removed element. Taking the image of the record carrier then yields ordinary set insertion and removal. The frontend checks the finite action laws and constructs the simulation. For Embedded RGA, the shared induction instead assumes a legal selected history and a one-step projection law. Proving insertion adjacency and deriving legality from the certified execution remain datatype-specific obligations.

Positional request preparation. For native RGA, Embedded RGA and its sided variant, clients request insertion or deletion at a visible position. Preparation supplies the anchor identity and, for

the embedded variants, the coordinate prefix. Let view erase coordinates while retaining the visible identities and values, and let edit_t be ordinary positional list editing, using t as the fresh insertion identity. At a certified source and an actual apply step, the checked local theorem is

$$\mathsf{prepare}(s, t, r, u) = e \quad \Longrightarrow \quad \mathsf{view}(\mathsf{do}(s, e)) = \mathsf{edit}_t(\mathsf{view}(s), u).$$

The theorem derives sortedness, identity uniqueness, positive coordinates and the required clock bound from the execution. It does not assume a legal sequential replay, or replace the separate global history bridge.

Retaining specification conflicts. For the stronger criterion of Definition 4.6, additionally require

$$a \leftrightarrow b \quad \Longrightarrow \quad f(a) \leftrightarrow_S f(b). \tag{2}$$

Here $\leftrightarrow$ is raw concrete commutation, as used in lo_E^C; the right-hand relation is contextual language commutation. By contraposition, every specification-conflicting visibility edge is an implementation-conflicting visibility edge. Thus the same canonical word also respects specification visibility. Equation (1) supplies admission; compatibility supplies only the additional order obligation. For an invariant-order variant, require the same implication with $\leftrightarrow_I$ in place of raw commutation; raw compatibility alone does not discharge that stronger premise. For full-event specifications take $f = \mathrm{id}$ and require the corresponding full-event fold premise. Erasing event metadata is an explicit specification choice, rather than a consequence of the bridge.

6.2 Choosing a permitted history within a certified execution

Global fold soundness can be unsuitable for a datatype whose public operations have guards: an arbitrary word may name a nonexistent birth, reuse an identity, or remove a live non-head. Restricting the implementation's representation invariant does not automatically prove the independent language's guards. A second route proves existence of a suitable history on the supported, causally closed event sets of certified executions.

Definition 6.3 (Execution-scoped history adequacy)**.** Fix an issuance discipline G, the raw implementation order lo_E^C, and an independent full-event language Spec; take $f = \mathrm{id}$. They satisfy execution-scoped history adequacy when, for every G-certified configuration C, every stored version (s, E) and every query q, there is a word π such that

$$\begin{aligned} &\pi \text{ enumerates } E \text{ exactly once,} \\ &\pi \text{ respects } \mathsf{lo}_E^C \text{ and } \mathrm{sv}_C^{f,\mathsf{Spec}}, \\ &\mathsf{ops}_f(\pi) \cdot [q \Rightarrow \mathsf{query}(\pi(\sigma_0), q)] \in \mathsf{Spec}. \end{aligned}$$

The quantifier order is $\forall q\, \exists \pi$. The accepted answer describes the merge-free fold, rather than the stored state s. Both order obligations and language admission use the same π.

Theorem 6.4 (Representation and history bridge)**.** *Fix the datatype, issuance discipline, independent language, and raw implementation replay order. Define exact concrete canonicality by*

$$\mathrm{Can}_{C,E}(s) \quad \Longleftrightarrow \quad \exists \pi.\ \pi \textit{ enumerates } E \ \wedge\ \pi \textit{ respects } \mathsf{lo}_E^C \ \wedge\ \pi(\sigma_0) = s.$$

This is an additional history-selection route, rather than a replacement for the manuscript's unrestricted lifting premise. The concrete specialization of the checked bridge has these assumptions:

1. *the raw replay order has a proved uniqueness property: any two exact canonical states for the same supported event set are equal;*
2. *every stored event set of the certified execution is supported, and its stored state is canonical for that set;*
3. *execution-scoped history adequacy of Definition 6.3 holds.*

Then every stored version satisfies the criterion with independent specification visibility: one word enumerates its original events, respects both required relations, and explains its stored query answer in Spec. *Consequently all active heads satisfy that criterion. If the assumptions are established at every configuration of a finite certified execution, the result holds throughout that execution, including executions with recursively synthesized virtual merge bases.*

Proof. Adequacy supplies a word π whose fold is canonical by its permutation and order properties. Replay uniqueness identifies that fold with the separately canonical stored state. Substitute this state equality in the accepted history. Its word, permutation and two order properties do not change. Apply the argument at each visited configuration for the execution claim. The stored-state premise is where the merge argument enters; history adequacy alone cannot discharge it. □

This uniqueness assumption must be established for the chosen implementation order; it is not automatic from scoped laws. For example, Theorem 4.9 requires causally ready implementation replays; it does not give equality of every pair of permutations respecting only the invariant order. A concrete port may instead prove an exact representation relation between its selected independent history and the stored live state, as in Example 6.5.

Example 6.5 (Anchored enqueue with an independent FIFO)**.** Enqueue records its issuer's observed tail and dequeue its observed head. The implementation keeps ordered live records and carried coordinates; it stores no removed birth records. The independent FIFO appends a fresh tagged value, ignoring its recorded tail; it removes a named head, permits repeated removal of an already removed identity, and rejects removal of a live non-head. The absent-removal case requires a prior birth in the sequential prefix; invented identities cannot be removed. Lack of tombstones does not mean lack of metadata: carried coordinates preserve placement information and can grow with their ancestry.

Clients need only request $\mathsf{enqueue}(v)$ or $\mathsf{dequeue}$. Preparation selects the live tail and its coordinate for enqueue, or the live head identity for dequeue; an empty dequeue generates no update event. Every message allowed by the original issuer arises from such a request. For a sorted issuing state with unique IDs and a fresh event timestamp, the prepared message satisfies the existing issuance predicate and its local effect on the value list is respectively $L \cdot [v]$ or $\mathsf{tail}(L)$. This local request theorem does not identify concurrent named-head removal with repeated destructive pops: two replicas may remove the same observed head.

Head-only issuance implies that removed births form a causal downset among enqueue events. Order the original dequeue events chronologically. Immediately before an identity's first dequeue, emit its birth; emit subsequent duplicate dequeues without another birth. Append the surviving births in their stored coordinate order. The FIFO is empty between the removal blocks. The downset property preserves causal enqueue conflicts, and surviving coordinates preserve the remaining causal enqueue order. This one word has the exact final tagged live list and satisfies both order obligations. The resulting theorem applies to certified ordinary and recursive virtual executions with the unchanged public tagged-head query.

Example 6.6 (FugueMax: positional clients and stable origins)**.** The client inserts at a position i or deletes the element at position i. Preparation interprets i in the issuing visible list. An insertion message records its fresh identity t and visible predecessor ℓ (zero at the front); a deletion records the

selected identity x. Structural parents, right origins and coordinate chains remain implementation metadata.

The independent sequential state is (B, T), a permanent identity list and set of deleted identities, initially $([0], \varnothing)$. Write $\mathrm{after}(B, \ell, t)$ for insertion immediately after ℓ. Its admitted transitions and read are

$$\begin{aligned}
(B,T) &\xrightarrow{\mathsf{insertAfter}(t,\ell)} (\mathrm{after}(B,\ell,t),T) \quad \text{when } t \neq 0 \wedge t \notin B \wedge \ell \in B,\\
(B,T) &\xrightarrow{\mathsf{delete}(x)} (B, T \cup (\{x\} \setminus \{0\})) \quad \text{when } x \in B,\\
\mathsf{read}(B,T) &= [x \in B \mid x \neq 0 \wedge x \notin T].
\end{aligned}$$

Deleted identities remain available as origins; the visible read omits them. The source issuer still chooses the requested visible gap. The checked local refinement proves that its actual physical update implements the positional edit. The current carrier observes identities; this theorem does not add an arbitrary character payload to that carrier.

For every stored version of an ordinary or recursively merged certified execution, the checked five-VC construction yields one permutation of its original events that respects the invariant-scoped implementation order and specification-conflicting visibility, is admitted by this language, and returns the stored query answer. The same witness carries the existing maximal noninterleaving property. The valid-state domain is unchanged and separately proved closed under updates and merges.

The earlier live-structural-parent language remains refuted by the certified insert–delete–insert trace: it confuses a structural tombstone with the client's visible predecessor. This obstruction does not apply to the language above. Requiring raw commutation on malformed unsorted lists would also impose a stronger implementation order than the established invariant criterion.

Native RGA's reserved root. Both insertion issuance and the identified sequential language require $t > 0$; zero denotes only the root. The certified no-root theorem derives exclusion of zero from visible lists for ordinary and recursive executions. This restores a source convention omitted in the earlier Lean guard, without changing the update or merge functions.

Example 6.7 (Published bounded counter)**.** Balegas et al., SRDS 2015, Section III-B, Figure 1, define a lower-bounded counter for a fixed set $I = \{0, \dots, n-1\}$ of replicas and bound $K \in \mathbb{Z}$. The state is $(R, U) \in \mathbb{N}^{I \times I} \times \mathbb{N}^I$, initially zero. The diagonal R_{ii} counts increments; R_{ij} for $i \neq j$ counts rights transferred from i to j; U_i counts decrements. Define

$$V(R,U) = K + \sum_{i \in I} R_{ii} - \sum_{i \in I} U_i, \qquad B_i(R,U) = R_{ii} + \sum_{j \in I \setminus \{i\}} R_{ji} - \sum_{j \in I \setminus \{i\}} R_{ij} - U_i.$$

Queries return V and B_i, respectively. For an event issued by $i \in I$ with amount $a \in \mathbb{N}$, the accepted updates are

$$\begin{aligned}
&\mathsf{do}((R,U), \mathsf{inc}(a), i) && = (R[(i,i) \mapsto R_{ii} + a], U), && \\
&\mathsf{do}((R,U), \mathsf{dec}(a), i) && = (R, U[i \mapsto U_i + a]), && a \leq B_i(R,U),\\
&\mathsf{do}((R,U), \mathsf{transfer}(a,j), i) && = (R[(i,j) \mapsto R_{ij} + a], U), && j \in I \setminus \{i\},\ a \leq B_i(R,U).
\end{aligned}$$

Only issuer i writes row i and component U_i. Its updates are serialized. The conditions on the right are original-issuance conditions; the effect of an accepted operation is the displayed arithmetic update. Nonnegative amounts make the source's monotonicity assumption explicit; the distinct recipient follows its instruction to transfer to another replica. Merge is

$$\mathsf{merge}(l, (R,U), (R',U')) = \big((\max(R_{ij}, R'_{ij}))_{i,j}, (\max(U_i, U'_i))_i\big).$$

Thus the ancestor is unused. Different donors write different cells even when they transfer to the same recipient.

An independent sequential machine has a scalar $v \in \mathbb{Z}$ and balances $b : I \to \mathbb{Z}$, initially $(K, 0)$. Its transitions are

$$\begin{aligned}
&(v,b) \xrightarrow{\mathsf{inc}(a),i} (v+a, b[i \mapsto b_i + a]), && i \in I, && \\
&(v,b) \xrightarrow{\mathsf{dec}(a),i} (v-a, b[i \mapsto b_i - a]), && i \in I, && a \le b_i, \\
&(v,b) \xrightarrow{\mathsf{transfer}(a,j),i} (v, b[i \mapsto b_i - a, j \mapsto b_j + a]), && i,j \in I,\ i \neq j, && a \le b_i.
\end{aligned}$$

The client value query returns v; the rights-accounting query returns b_i. The projection is $(R, U) \mapsto (V(R,U), B(R,U))$. Legal sequential runs preserve $v = K + \sum_i b_i$ and $b_i \ge 0$, hence $v \ge K$.

The five merge equations are checked with visibility as metadata dependence. For the causal-delta equation, same-replica visibility ensures that the issuing row in the remainder equals its row in the causal past; other rows dominate the past componentwise. The other four equations follow from maximum algebra. The checked sequential-history bridge transports original issuance funds to a causal replay prefix: that prefix contains every prior debit from the issuer, and additional events can only contribute incoming credits. Every certified stored version therefore has a legal explaining history and satisfies the local lower bound. The same argument applies to the finite union E of all issued events, whether or not any stored version contains this union:

$$K + \sum_{e \in E} \Delta(e) \ge K, \qquad \Delta(\mathsf{inc}(a)) = a, \quad \Delta(\mathsf{dec}(a)) = -a, \quad \Delta(\mathsf{transfer}(a,j)) = 0.$$

Finiteness of E is derived from execution. Both ordinary executions and executions with recursively reconstructed merge bases are covered. The public history, local-safety and global-bound theorem dependencies are audited to include all five VCs and the generic merge induction.

Example 6.8 (Neem enable-wins and disable-wins flags)**.** Neem's corrected flag stores one counter–bit pair $(c_i, b_i) \in \mathbb{Z} \times \{\mathsf{false}, \mathsf{true}\}$ per replica i. Its source constant maps have universal domain. The two implementations share physical operations W (the winning update) and Z (the resetting update):

$$\mathsf{do}(s,(t,i,W))(j) = \begin{cases} (c_i + 1, \mathsf{true}) & j = i, \\ s(j) & j \neq i, \end{cases} \qquad \mathsf{do}(s,(t,i,Z))(j) = (c_j, \mathsf{false}).$$

For each row, merge retains Neem's ancestor subtraction and bit test:

$$c_m = c_a + c_b - c_l, \qquad b_m = \begin{cases} \mathsf{true} & b_a \wedge b_b, \\ \mathsf{false} & \neg b_a \wedge \neg b_b, \\ c_a > c_l & b_a \wedge \neg b_b, \\ c_b > c_l & \neg b_a \wedge b_b. \end{cases}$$

The source operation names, initial rows, public query and policy are

	W	Z	$s_0(i)$	$\mathsf{read}(s)$	rc
enable-wins	enable	disable	$(0, \mathsf{false})$	$\exists i.\ b_i$	disable $\to$ enable
disable-wins	disable	enable	$(0, \mathsf{true})$	$\neg\exists i.\ b_i$	enable $\to$ disable

Neem's F* application files specify updates and merge but no read operation. The enable-wins query follows the paper's surviving-enable interpretation; the disable-wins query is its Boolean

dual. Both resulting public flags start false. The independent sequential state is a Boolean v, initially false, with $v \xrightarrow{\mathsf{enable}} \mathsf{true}$, $v \xrightarrow{\mathsf{disable}} \mathsf{false}$, and read returning v. All operations are issuable; execution still serializes each replica's updates.

The proof represents each counter by its number of winning events. A row bit is true when a winning event from that replica survives every visible reset, or when its initially true bit has encountered no reset. Full causal metadata retains the single-writer prefixes. The source merge equations are proved from this representation; the implementation remains the displayed ternary merge. Both flags have checked five-VC certificates and independent Boolean-history bridges for ordinary and recursive-merge executions. The dependency audit checks that both public certificates consume the five VCs and generic merge induction; nine PASS+FAIL examples exercise the actual Lean definitions.

A Correspondence with the Manuscript

This table identifies the manuscript location to revise and the corresponding statement in this reference. Numbers in the left column refer to Overleaf `0450d48`; numbers in the right column refer to this PDF. The following appendix links each reference statement to its Lean declarations. Thus a reader can follow a manuscript definition to the mathematical replacement and then, if needed, to the checked source.

Manuscript	Reference and relationship
Section 3, configuration and Apply/Merge rules	Definition 3.1 retains $C = \langle S, H, P, \mathsf{vis}\rangle$ and the same transition updates, making rank, clock and fresh-fork premises explicit. Definition 4.7 adds original issuance evidence.
Lemma 3.1, LCA	Definition 3.2 fixes the intended *greatest* ancestor; Theorem 3.3 supplies the event-origin hypothesis behind the intersection argument. Definition 3.4 is an extension for multiple merge bases.
Section 4, Notations	Definition 4.1 keeps event replay and the operation policy. Definition 4.2 adds a justified invariant to the commutation domain; implementation equality remains concrete.
Definitions 4.1–4.2, linearization relation and linear extension	Definition 4.3 keeps the two clauses and local absorber; the same invariant domain is used in both noncommutation tests.
Definition 4.3, sequential specification	Definition 4.4 retains the language generated by a labelled transition system and allows an explicit event-to-label map. Specification conflict is an additional definition on that language.
Section 4.1, three replay laws	Definition 4.5 separates operation-distinct exactness from equal-operation self-commutation, retaining the event guards. Definitions 4.7 and 4.8 state the distinct certified alternative; its scope must not be omitted.
Definition 4.4, RA-linearizability	Definition 4.6 is a substantive extension: one witness must also preserve specification-conflicting visibility. The preceding paragraph states the literal manuscript criterion separately.

Manuscript	Reference and relationship
Theorem 4.5; Definitions 4.6–4.7, canonical states	Section 4's canonical-state discussion and Theorem 4.9 distinguish global linear-extension uniqueness on supported, causally closed sets from scoped legal-replay uniqueness. Existence is an explicit obligation.
Section 5.1, WeakClosed, downset, Admissible, VC1–VC5	Definitions 5.1–5.4 state the representation, metadata dependencies, equations and reconstruction supplies. These are hypotheses, not consequences of query agreement.
Lemma 5.1, Join	Theorem 5.5 proves the union representation by peeling. Canonical join follows after relating representation to canonical replay.
Theorem 5.2, canonical correctness	Theorem 5.7 lifts a canonical join contract to certified executions and recursive virtual bases. It states the internal replay order separately from the public history order.
Section 6, broken set; Theorem 6.1 and its simulation premise	Example 6.1 distinguishes literal RA from the strengthened criterion. Theorem 6.2 gives the literal lifting step; Equation (2) supplies a sufficient extra condition for specification visibility. The simulation uses the paper's specification-refinement function α.
Extension for guarded datatypes	Definition 6.3 and Theorem 6.4 replace unrestricted fold soundness by a certified history argument with explicit concrete-state uniqueness. Examples 6.5–6.6 show its scope and limitations.

Presentation changes and mathematical changes. The section order, notation and names in this revision follow the manuscript. The transition system and datatype implementations have not been changed by this rewrite. The invariant commutation domain, original issuance evidence, metadata reconstruction premises and specification-visibility clause are mathematical differences from the manuscript, already present in the checked baseline. They require explicit manuscript changes rather than a renaming. The specification function α in Section 6 relates concrete states to states of the independent sequential specification.

B Mechanization Traceability

The main text can be read without Lean. This appendix maps each numbered statement to its checked declarations. Some statements group declarations or specialize a generic interface to concrete equality. Unchanged source links use the proof baseline; migrated and new declarations link to their accompanying files on `paper1`. The declaration-check script supplied with this reference checks that every name resolves. The theorem ledger separately audits final roots for permitted axioms. For Definitions 4.5 and 4.8, the production split interfaces below state operation-distinct exactness and equal-operation self-commutation separately. Checked equivalences retain the previous combined contracts on exactly the same event and state domains.

Reference 3.1.
Execution.lean:

`Sal.MRDTs.Configuration`
`Sal.MRDTs.Step`

Signature.lean:

`Sal.MRDTs.MRDTSig`

Reference 3.2.

Execution.lean:

`Sal.MRDTs.IsGCA`

Reference 3.3.

StoreInvariant.lean:

`Sal.MRDTs.gca_events_of_storeInv`

Reference 3.4.

VirtualMergeBase.lean:

`Sal.MRDTs.virtualMergeBaseState`

`Sal.MRDTs.vfoldAux`

Execution.lean:

`Sal.MRDTs.StepV`

Reference 4.1.

UpdateSignature.lean:

`Sal.MRDTs.Foundation.Op`

`Sal.MRDTs.Foundation.UpdateSig.commutes`

RestrictedReplay.lean:

`Sal.MRDTs.Paper1.OperationPolicy`

`Sal.MRDTs.Paper1.OperationPolicy.lift_replayOrder_iff`

HistoricalReplay.lean:

`Sal.MRDTs.Foundation.distinctOps`

Reference 4.2.

InvariantOrder.lean:

`Sal.MRDTs.Paper1.InvariantOrder.Commutes`

`Sal.MRDTs.Paper1.InvariantOrder.Closed`

Reference 4.3.

InvariantOrder.lean:

`Sal.MRDTs.Paper1.InvariantOrder.order`

`Sal.MRDTs.Paper1.InvariantOrder.order_universal`

RestrictedReplay.lean:

`Sal.MRDTs.Paper1.paperOrder`

Replay.lean:

`Sal.MRDTs.Foundation.listPermOf`

`Sal.MRDTs.Foundation.respects`

Reference 4.4.

HistorySpec.lean:

`Sal.MRDTs.Paper1.HistorySpec`

`Sal.MRDTs.Paper1.HistoryMachine.toSpec`

SpecificationVisibility.lean:

`Sal.MRDTs.Paper1.HistorySpec.Commutes`

ProjectedCriterion.lean:

`Sal.MRDTs.Paper1.projectedLabels`

`Sal.MRDTs.Paper1.projectedSpecVisibility`

Reference 4.5.
SplitPolicy.lean:
- `Sal.MRDTs.Paper1.SplitPolicy.Laws`
- `Sal.MRDTs.Paper1.SplitPolicy.laws_iff_guarded`

Reference 4.6.
InvariantOrder.lean:
- `Sal.MRDTs.Paper1.InvariantOrder.VersionsRA`
- `Sal.MRDTs.Paper1.InvariantOrder.versions_universal`

EventBridge.lean:
- `Sal.MRDTs.Paper1.EventVersionsSpecificationRA`
- `Sal.MRDTs.Paper1.EventVersionsSpecificationRA.heads`
- `Sal.MRDTs.Paper1.EventSequentialSimulation`

ProjectedCriterion.lean:
- `Sal.MRDTs.Paper1.ProjectedSpecificationRALinearizable`

RALinearizability.lean:
- `Sal.MRDTs.Paper1.RALinearizable`
- `Sal.MRDTs.Paper1.RAExecution`
- `Sal.MRDTs.Paper1.RAImplementation`

Reference 4.7.
Certificates.lean:
- `Sal.MRDTs.MintHonest`
- `Sal.MRDTs.Issuance`
- `Sal.MRDTs.IssuedStep`
- `Sal.MRDTs.MintCertifiedReach`
- `Sal.MRDTs.MintCertifiedReachV`

Correctness.lean:
- `Sal.MRDTs.CertifiedExecution`

CertifiedReplay.lean:
- `Sal.MRDTs.Paper1.CertifiedReplay.Scope`
- `Sal.MRDTs.Paper1.CertifiedReplay.Ready`
- `Sal.MRDTs.Paper1.CertifiedReplay.Legal`

Reference 4.8.
InvariantReplay.lean:
- `Sal.MRDTs.Paper1.InvariantReplay.Laws`

SplitPolicy.lean:
- `Sal.MRDTs.Paper1.SplitPolicy.Invariant.PolicyLaws`
- `Sal.MRDTs.Paper1.SplitPolicy.Invariant.RestrictedLaws`
- `Sal.MRDTs.Paper1.SplitPolicy.Invariant.restricted_iff_existing`
- `Sal.MRDTs.Paper1.SplitPolicy.Certified.restricted_iff_existing`

CertifiedPolicy.lean:
- `Sal.MRDTs.Paper1.CertifiedReplay.Commutes`

Reference 4.9.
InvariantReplay.lean:
- `Sal.MRDTs.Paper1.InvariantReplay.convergence_on`
- `Sal.MRDTs.Paper1.InvariantReplay.Canonical`
- `Sal.MRDTs.Paper1.InvariantReplay.canonical_unique`

ConcreteReplay.lean:

`Sal.MRDTs.Paper1.ConcreteMRDT.canonical_unique`

ConcreteFormalism.lean:

`Sal.MRDTs.Paper1.ConcreteMRDT.Canonical`

Reference 5.1.

ConcreteFormalism.lean:

`Sal.MRDTs.Paper1.ConcreteMRDT.Representation`

ConcreteJoin.lean:

`Sal.MRDTs.Paper1.ConcreteMRDT.Raw.Unique`

Reference 5.2.

ConcreteMetadata.lean:

`Sal.MRDTs.Paper1.ConcreteMRDT.MetadataDependencies`

`Sal.MRDTs.Paper1.ConcreteMRDT.MetadataDependencies.Closed`

`Sal.MRDTs.Paper1.ConcreteMRDT.MetadataDependencies.Past`

`Sal.MRDTs.Paper1.ConcreteMRDT.MetadataDependencies.closed_diff_of_max`

Reference 5.3.

ConcreteJoin.lean:

`Sal.MRDTs.Paper1.ConcreteMRDT.Raw.Context`

`Sal.MRDTs.Paper1.ConcreteMRDT.Raw.MergeVCs`

RestrictedReplay.lean:

`Sal.MRDTs.Paper1.paperOrder`

Reusable verification templates (Section 5.1).

CommonVerification.lean:

`Sal.MRDTs.Paper1.Automation.CommonVerification.Input`

`Sal.MRDTs.Paper1.Automation.CommonVerification.verify`

InputDerivation.lean:

`Sal.MRDTs.Paper1.Automation.InputDerivation.PolicyData`

`Sal.MRDTs.Paper1.Automation.InputDerivation.MaskData`

MaskKitAutomation.lean:

`Sal.MRDTs.Paper1.Automation.MaskKitAutomation.Description`

Reference 5.4.

ConcreteJoin.lean:

`Sal.MRDTs.Paper1.ConcreteMRDT.Raw.PeelChoice`

`Sal.MRDTs.Paper1.ConcreteMRDT.Raw.ReplaySupply`

Reference 5.5.

ConcreteJoin.lean:

`Sal.MRDTs.Paper1.ConcreteMRDT.Raw.join_at_sizes`

`Sal.MRDTs.Paper1.ConcreteMRDT.Raw.representationJoin_of_vcs`

Reference 5.7.

CertifiedClosedExecution.lean:

`Sal.MRDTs.Paper1.CertifiedClosedExecution.ClosedJoinAt`

`Sal.MRDTs.Paper1.CertifiedClosedExecution.canonicalConfig_of_execution`

`Sal.MRDTs.Paper1.CertifiedClosedExecution.virtualMergeBaseState_canonical`

CertifiedRGAExecution.lean:

`Sal.MRDTs.Paper1.CertifiedRGAExecution.Embedded.represented_of_canonical`

Sal.MRDTs.Paper1.CertifiedRGAExecution.Embedded.canonical_of_represented
Sal.MRDTs.Paper1.CertifiedRGAExecution.Embedded.closedJoinAt

SetRelativeReplay.lean:

Sal.MRDTs.Foundation.loOn
Sal.MRDTs.Foundation.IsCanonicalState

Reference 6.1.

CriterionCounterexample.lean:

Sal.MRDTs.Paper1.CriterionCounterexample.mutant_rawRA
Sal.MRDTs.Paper1.CriterionCounterexample.criteria_differ_on_reachable_mutation

Reference 6.2.

PaperPresentation.lean:

Sal.MRDTs.Paper1.PaperPresentation.uniform_versionsRA_of_paperOrder_witness

Reference 1.

SequentialSimulation.lean:

Sal.MRDTs.Paper1.SequentialSimulation.sound
Sal.MRDTs.Paper1.SequentialSimulation

FiniteSetSimulation.lean:

Sal.MRDTs.Paper1.Automation.FiniteSetDescription.project_update

GenericGuardedFold.lean:

Sal.MRDTs.Paper1.Automation.guarded_fold_projection

RALinearizability.lean:

Sal.MRDTs.Paper1.ra_of_canonical_total

Reference 6.1.

SequentialAbstraction.lean:

Sal.MRDTs.Paper1.DeterministicSpec.Abstraction.admits_iff

SimpleEventPorts.lean:

Sal.MRDTs.Paper1.SimpleEventPorts.set_language_preserved
Sal.MRDTs.Paper1.SimpleEventPorts.finite_language_preserved
Sal.MRDTs.Paper1.SimpleEventPorts.boolean_language_preserved
Sal.MRDTs.Paper1.SimpleEventPorts.delta_language_preserved

Reference 6.1.

EmbedRGAClientLocal.lean:

Sal.MRDTs.Paper1.EmbedRGAClient.certified_prepared_physical

SidedRGAClientLocal.lean:

Sal.MRDTs.Paper1.SidedRGAClient.certified_prepared_physical

RGAClientPhysical.lean:

Sal.MRDTs.Paper1.RGA.Client.Physical.certified_query_prepared
Sal.MRDTs.Paper1.RGA.Client.Physical.certified_query_preparedV

Reference 2.

CommutationBridge.lean:

Sal.MRDTs.Paper1.CommutationCompatibility
Sal.MRDTs.Paper1.projectedSpecVisibility_sub_paperOrder
Sal.MRDTs.Paper1.specificationRA_of_fiveVCs_compatible

ProjectedSimulation.lean:

Sal.MRDTs.Paper1.MachineProjection.language_commutes

Reference 6.3.
ConcreteHistoryBridge.lean:
`Sal.MRDTs.Paper1.ConcreteMRDT.ExecutionHistoryAdequacy`

Reference 6.4.
ConcreteHistoryBridge.lean:
`Sal.MRDTs.Paper1.ConcreteMRDT.versions_of_scoped_history`
ConcreteVCExecution.lean:
`Sal.MRDTs.Paper1.ConcreteMRDT.ScopedVCConditions.executionsV`

Reference 6.5.
AnchoredQueueClient.lean:
`Sal.MRDTs.Paper1.AnchoredQueue.Client.prepared_canIssue`
`Sal.MRDTs.Paper1.AnchoredQueue.Client.prepared_refines`
`Sal.MRDTs.Paper1.AnchoredQueue.Client.canIssue_iff_prepared`
AnchoredQueueCausality.lean:
`Sal.MRDTs.Paper1.AnchoredQueue.removed_births_causal_downset`
AnchoredQueueCertificate.lean:
`Sal.MRDTs.Paper1.AnchoredQueue.History.stored_publicWitness`
`Sal.MRDTs.Paper1.AnchoredQueue.History.correctV`
AnchoredQueue.lean:
`Sal.MRDTs.Paper1.AnchoredQueue.fifoLegal`
`Sal.MRDTs.Paper1.AnchoredQueue.CanIssue`
EmbedRGA.lean:
`Sal.MRDTs.Instances.EmbedRGA.eCoord`
Code.lean:
`Sal.EmbedRGA.unaryEnc`

Reference 6.6.
CertifiedFugueInvariantObstruction.lean:
`Sal.MRDTs.Paper1.CertifiedFugueInvariantObstruction.certified_failure`
`Sal.MRDTs.Paper1.CertifiedFugueInvariantObstruction.implementation_spec_separation`
FugueMaxNaturalSequence.lean:
`Sal.MRDTs.Paper1.FugueMaxNaturalSequence.machine`
FugueMaxHonestPhysicalRefinement.lean:
`Sal.MRDTs.Paper1.FugueMaxHonestPhysicalRefinement.honest_physical_insert_refinement`
FugueMaxNaturalCertificate.lean:
`Sal.MRDTs.Paper1.FugueMaxNaturalCertificate.version_witness_noninterleaving`
`Sal.MRDTs.Paper1.FugueMaxNaturalCertificate.correct`
`Sal.MRDTs.Paper1.FugueMaxNaturalCertificate.correctV`

Reference 6.2.
RGAClientLocal.lean:
`Sal.MRDTs.Paper1.RGA.Client.certified_no_root`

Reference 6.7.
BalegasBoundedCounter.lean:
`Sal.MRDTs.Instances.BalegasBoundedCounter.localRights`
BalegasBoundedCounterVC.lean:
`Sal.MRDTs.Paper1.BalegasBoundedCounterVC.fiveVC`

BalegasBoundedCounterCertificate.lean:

`Sal.MRDTs.Paper1.BalegasBoundedCounterEvent.certifiedV`

`Sal.MRDTs.Paper1.BalegasBoundedCounterEvent.versions_safeV`

BalegasBoundedCounterGlobal.lean:

`Sal.MRDTs.Paper1.BalegasBoundedCounterEvent.global_boundV`

Reference 6.8.

EnableWinsFlagVC.lean:

`Sal.MRDTs.Paper1.NeemFlagVC.fiveVC`

EnableWinsFlagCertificate.lean:

`Sal.MRDTs.Paper1.EnableWinsFlagEvent.certifiedV`

DisableWinsFlagCertificate.lean:

`Sal.MRDTs.Paper1.DisableWinsFlagEvent.certifiedV`